

From 0.60 to 0.92: My Radar Segmentation Journey

Author: Sergo — a concise engineering story about moving a radar segmentation baseline (0.60) past a 0.90 benchmark through disciplined modeling, instrumentation, and data work.



Slide 1 — The Challenge

Problem

Semantic segmentation on radar returns — noisy, sparse, and physics-driven.

Baseline

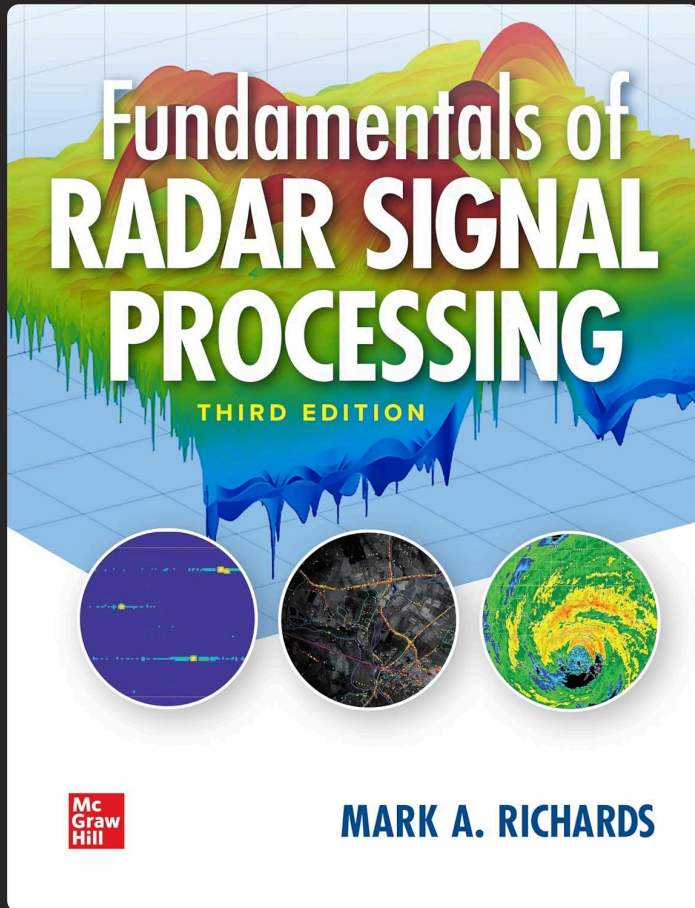
Score = **0.60**
(simple CNN; no modern training practices)

Target

Committee benchmark = **0.90**. Goal: meet or exceed it.

Approach

Build intuition, iterate architecture, then fix data.



Slide 2 — Understanding the Domain (and My Limitation)

I spent time on readings and videos to understand radar returns: signal propagation, speckle, and range-azimuth artifacts. It became clear that engineering perfect hand-crafted features would need deep domain expertise.

Decision: optimize what I could control — model architecture, training discipline, and later data augmentation — instead of trying to solve radar physics from scratch.

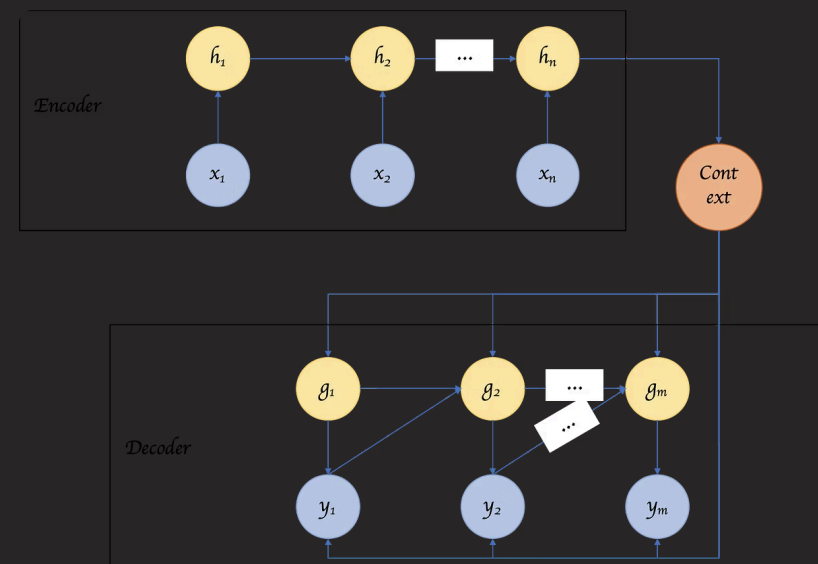
Slide 3 — Discovering Semantic Segmentation

I shifted from ad-hoc CNN stacks to structured encoder–decoder designs.

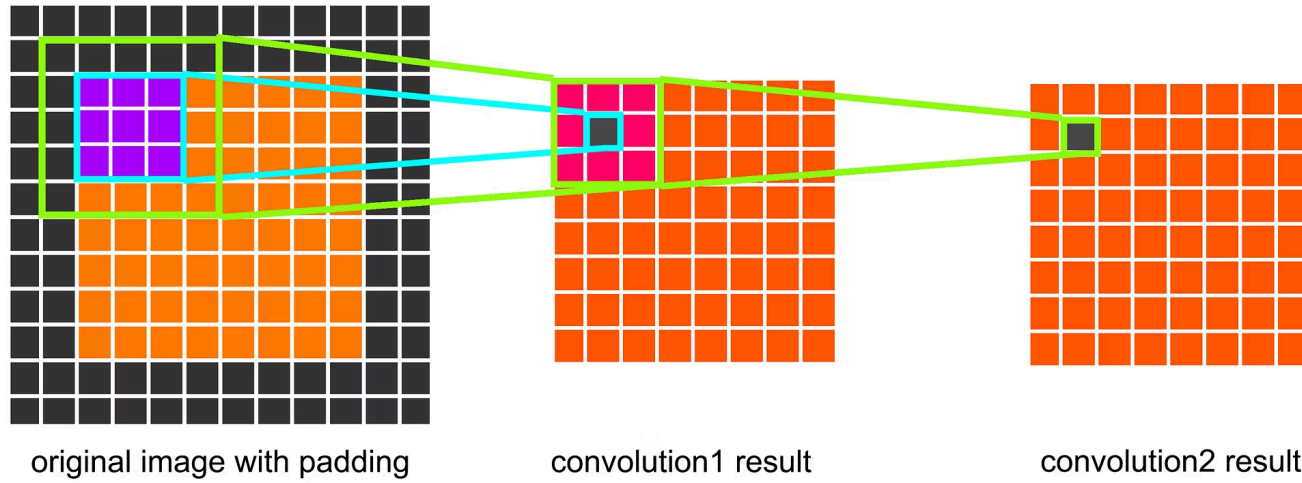
Core idea:

- Encoder → compress spatial detail into feature representation
- Decoder → reconstruct pixel-wise segmentation map

This reframed architecture design and clarified where to add inductive bias (skip connections, pooling/unpooling choices, and upsampling strategy).



Slide 4 — Improving the Baseline



Baseline

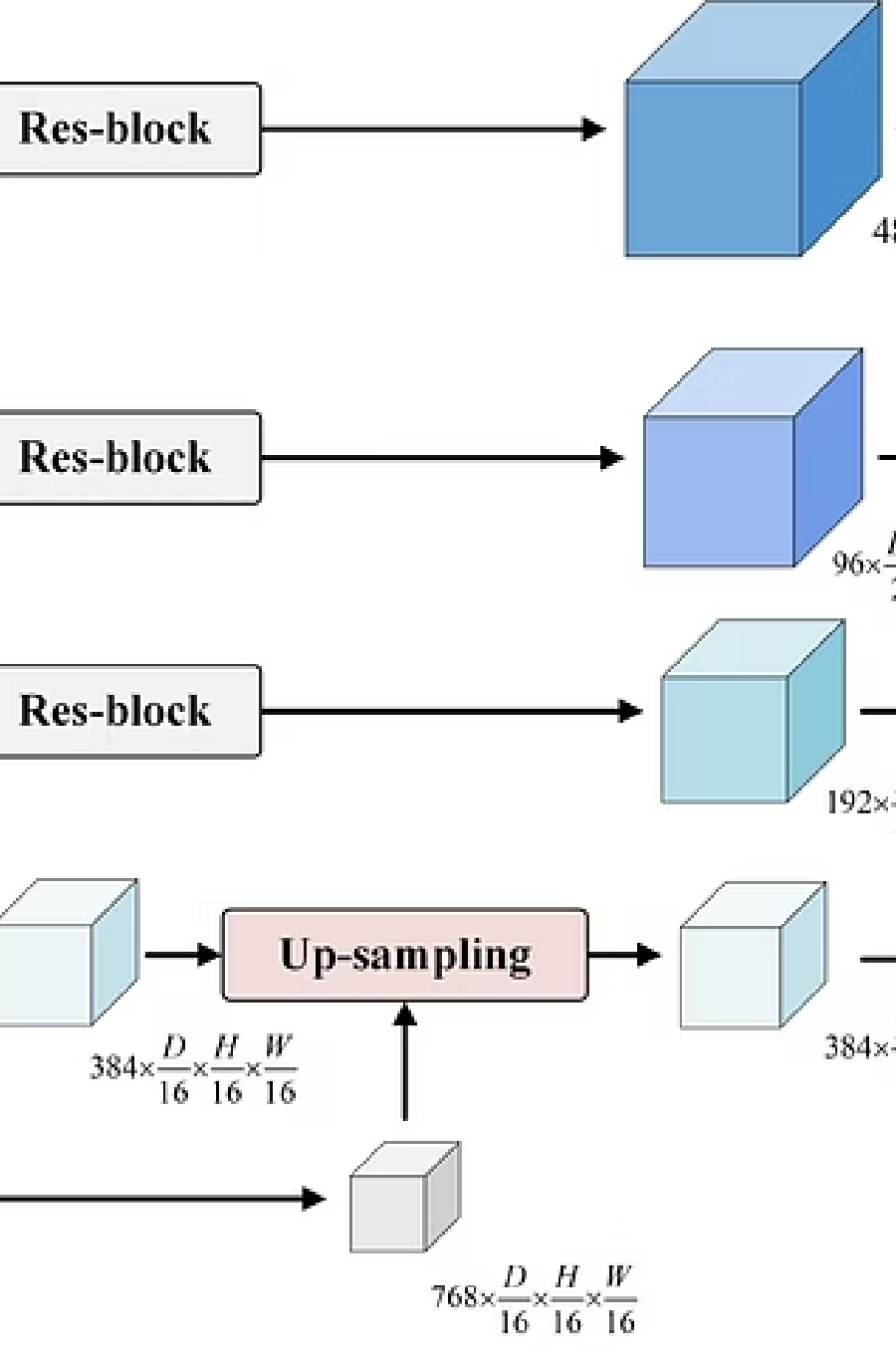
3 conv layers, no activations, no normalization → Score: **0.60**

First fixes

Added ReLU + BatchNorm.

Result

Score improved to **0.72**. Lesson: basic architectural hygiene matters.



Slide 5 — Choosing SegNet

I evaluated several segmentation families and implemented a simplified SegNet because it matched my constraints: clear encoder-decoder flow, feasible to implement, and intuitive debugging via pooling indices.

Why SegNet

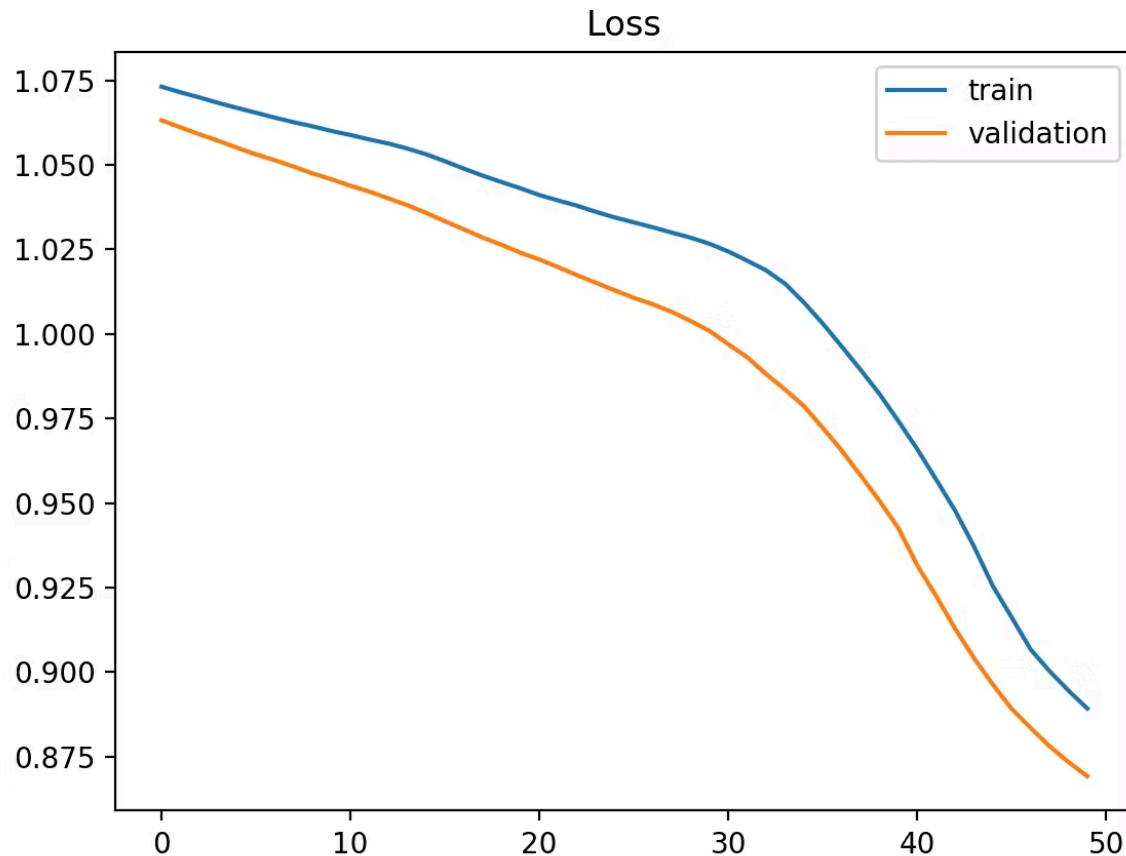
Clean conceptual mapping from encoder to decoder. Easier to reason about spatial information retention.

Personal note

"Seg" resonates — a small, human tie to the work.

Outcome

After implementing simplified SegNet
→ Score: **0.88**
(near the 0.90 target).



Slide 6 — Trying to Break the 0.90 Barrier

Optimization experiments:

- Increased epochs (40 → 100): loss plateaued around epoch ~50.
- Added LR scheduler (patience=3, factor=0.1): resumed learning but no significant score uplift.

Conclusion: optimization tweaks improved training behavior but did not change the test generalization ceiling.

Slide 7 — Identifying the Real Bottleneck

Key insight: dataset size and diversity were the limiting factor. The model had capacity but lacked representative data — classic overfitting ceiling.

Observation

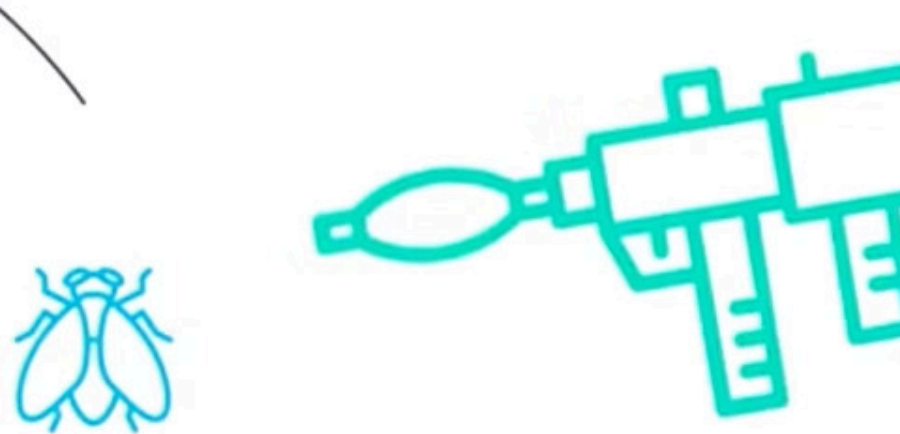
High variance between train and validation performance; small dataset magnified this effect.

Shift in strategy

Move from architecture-first to data-first: augment and diversify inputs.

OVERFITTING

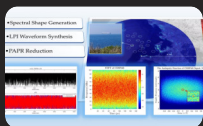
Error due to variance
model performs poorly in the testing



Slide 8 — Data Augmentation Breakthrough

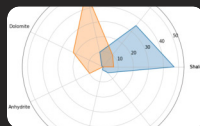
Experiments:

- Initial: torchvision.transforms — instability on radar signals.
- Final: Albumentations pipeline with targeted transforms:



Gaussian Noise

Improves robustness to sensor noise and small amplitude variations.



Brightness / Contrast

Simulates reflectivity variability and preprocessing differences.



Geometric transforms

Adds spatial diversity while preserving physical plausibility.

Result: final score reached **0.92** — above the 0.90 benchmark. Data diversity unlocked generalization.

Slide 9 — What I Learned

Architectural fundamentals

Encoder-decoder structure is core for segmentation tasks.

Optimization tools

LR schedulers and proper training hygiene are essential but not magic.

Data matters most

Augmentation and diversity often beat adding depth.

Practical engineering

MaxUnpool2d quirks → prefer ConvTranspose2d for cleaner upsampling in practice.

Final thought

Engineering is iterative diagnosis: identify the real bottleneck, then fix it deliberately.

Final arc: 0.60 → 0.72 → 0.88 → 0.92. The win came from combining disciplined architecture work with decisive, targeted data improvements.